

Memory- and Communication-Aware Model Compression for Distributed Deep Learning Inference on IoT

KARTIKEYA BHARDWAJ, CHING-YI LIN, ANDERSON SARTOR, and RADU MARCULESCU, Carnegie Mellon University, USA

Model compression has emerged as an important area of research for deploying deep learning models on Internet-of-Things (IoT). However, for extremely memory-constrained scenarios, even the compressed models cannot fit within the memory of a single device and, as a result, must be distributed across multiple devices. This leads to a *distributed inference* paradigm in which memory and communication costs represent a major bottleneck. Yet, existing model compression techniques are not communication-aware. Therefore, we propose *Network of Neural Networks* (NoNN), a new distributed IoT learning paradigm that compresses a large pre-trained ‘teacher’ deep network into several disjoint and highly-compressed ‘student’ modules, without loss of accuracy. Moreover, we propose a network science-based knowledge partitioning algorithm for the teacher model, and then train individual students on the resulting disjoint partitions. Extensive experimentation on five image classification datasets, for user-defined memory/performance budgets, show that NoNN achieves higher accuracy than several baselines and similar accuracy as the teacher model, while using minimal communication among students. Finally, as a case study, we deploy the proposed model for CIFAR-10 dataset on edge devices and demonstrate significant improvements in memory footprint (up to 24×), performance (up to 12×), and energy per node (up to 14×) compared to the large teacher model. We further show that for distributed inference on multiple edge devices, our proposed NoNN model results in up to 33× reduction in total latency w.r.t. a state-of-the-art model compression baseline.

CCS Concepts: • **Computing methodologies** → **Object recognition; Neural networks**; • **Computer systems organization** → **Embedded systems**; • **Theory of computation** → *Distributed algorithms*;

Additional Key Words and Phrases: Network of neural networks, model compression, communities

ACM Reference format:

Kartikeya Bhardwaj, Ching-Yi Lin, Anderson Sartor, and Radu Marculescu. 2019. Memory- and Communication-Aware Model Compression for Distributed Deep Learning Inference on IoT. *ACM Trans. Embed. Comput. Syst.* 18, 5s, Article 82 (October 2019), 22 pages.
<https://doi.org/10.1145/3358205>

1 INTRODUCTION

Even though deep learning has gained significant importance, it is challenging to implement these models on resource-constrained devices. Such devices are commonly used in the ever-growing *Internet-of-Things* (IoT) domain. For instance, microcontrollers such as Arm Cortex-M which is

This article appears as part of the ESWEET-TECS special issue and was presented in the International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS), 2019.

Authors’ addresses: K. Bhardwaj, C.-Y. Lin, A. Sartor, and R. Marculescu, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA, USA, 15213; emails: {kbhardwa, cylin, asartor, radum}@cmu.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Association for Computing Machinery.

1539-9087/2019/10-ART82 \$15.00

<https://doi.org/10.1145/3358205>

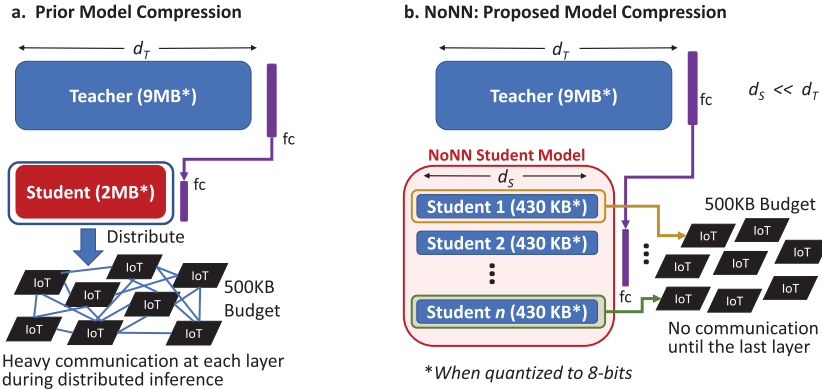


Fig. 1. (a) Prior art: Distributing large student models that do not fit on a single memory-limited IoT device leads to heavy communication at each layer. For instance, a student model that requires 2MB total memory cannot fit within an IoT-device which has only 500KB memory budget. (b) Proposed NoNN results in several disjoint students that can fit on individual IoT-devices: No communication until the very final layer (since NoNN students that require 430KB memory can fit within the 500KB budget).

used in many IoT applications such as Smart Healthcare, Keyword Spotting, *etc.*, has only 500KB available memory [27]. For such devices, prominent deep networks like AlexNet/Resnets that use up to 60M parameters are unsuitable. Therefore, there is a fundamental need for highly-efficient compressed deep learning models to enable *fast and computationally inexpensive inference* on the resource-constrained IoT-devices.

To this end, several deep learning model compression techniques exist in the literature. The most common approaches are pruning [5, 23], quantization [8, 12], and Knowledge Distillation (KD) [1, 6] and its variants such as Attention Transfer (AT) [25]. Since deep models can learn a large number of redundant or useless weights/channels, pruning aims to remove such parameters without sacrificing accuracy [5, 23]. Moreover, quantization reduces the number of bits required to represent weights and/or activations in deep networks [8, 12]. Also, KD-based approaches rely on teacher-student training, where a teacher model is a large deep network we want to compress. KD trains a significantly smaller student model with far fewer layers/width, to mimic the teacher network (see Figure 1(a)). Further, most prior art on model compression refers to pruning and quantization. However, since KD can lead to significantly compressed models, we assume that KD is also a model compression technique throughout this paper. Of note, the present work considers model compression and distributed inference for Convolutional Neural Networks (CNNs) used in image classification problems. However, distributed inference for other deep learning models such as Recurrent Neural Networks (RNNs) for speech/natural language applications can also be explored in a future work.

The existing approaches mentioned above *cannot* be used for *extremely memory-constrained* IoT scenarios (e.g., microcontrollers with a *total* memory of 500KB [20, 27]). Yet, a lot of smart home/cities applications can have several such connected but resource-limited devices. To achieve higher accuracy, pruning- or KD-based student models often grow in size due to which such models cannot fit on an individual IoT-device and must be distributed across multiple devices; this distribution of computation on multiple devices generates significant overhead in communication. For example, Figure 1(a) illustrates KD with a teacher model containing around 9 million parameters, and a large student model containing about 2 million parameters. Also, suppose that we are trying to deploy the student model (which will require 2MB memory after quantization) on IoT-devices

with only 500KB memory budget. Distributing this student model on multiple such IoT-devices will incur heavy communication at each of its intermediate layers. Consequently, in addition to the computation cost, a major (and largely ignored) impediment for widespread deployment of deep learning on IoT is this *communication* cost during distributed inference, which occurs due to extremely limited memory in IoT environments.

Therefore, we need to answer the following **key question**: *How can we compress a given deep learning model into multiple separate modules that: (i) fit within the memory and performance budgets per device, (ii) minimize the communication latency when distributed across a network of edge devices, and (iii) achieve high accuracy?*

To address this question, in this paper, we propose *Network of Neural Networks* (NoNN), a paradigm to derive low memory-, computation-, as well as communication-cost student architectures from a single (powerful) but much larger teacher model. As shown in Figure 1(b), a NoNN consists of multiple, *memory-limited* student networks which individually learn only a specific part of teacher's function. Indeed, training an individual student to mimic only a part of teacher's function is a significantly simpler problem than mimicking its entire function. Since this highly-parallel architecture of NoNN allows us to effectively increase the overall model-size, we obtain higher accuracy without significantly increasing the communication costs among the disjoint students. These individual students can then be deployed on separate resource-constrained IoT-devices to perform the distributed inference (see Figure 1(b)). This can ultimately help preserve user's privacy by conducting inference on a network of edge devices instead of sending data to the cloud.

Finally, to distribute knowledge from a teacher to multiple students, we propose a new approach based on network science [16, 17]. Specifically, as our objective is to obtain individual student models below certain memory-constraints, network science allows for significant flexibility in terms of how the knowledge from the teacher is distributed across multiple students. For instance, with our proposed network-theoretic approach, we uniformly distribute knowledge from the teacher across the students to achieve high accuracy.

Overall, we make the following **key contributions**:

- (1) Since there has been limited research on performing efficient distributed inference on highly memory-constrained IoT devices, we propose a new NoNN paradigm to compress a large teacher into a set of independent and highly-parallel student networks. In prior works, student models that cannot fit on a single device can lead to significant communication at all layers, whereas NoNN communicates only at the last layer. This makes NoNN an ideal candidate for distributed deep learning. To the best of our knowledge, we are the *first* to introduce a communication-aware model compression for distributed inference on IoT-devices.
- (2) We are also the first to formulate the model compression problem from a network science angle. By building a network of filter activation patterns, we exploit principles from network science such as community detection [17] to split teacher's knowledge into multiple disjoint partitions which, in turn, results in disjoint and compressed student modules.
- (3) Extensive experiments on five well-known image classification tasks demonstrate that NoNN achieves significantly lower memory (2.5×-24× reduction w.r.t. teacher) and computation (2×-15× fewer FLOPS w.r.t. teacher), with minimal communication costs and similar accuracy as the teacher model. Also, NoNN achieves higher classification accuracy than prior art [6, 10, 25].
- (4) We further deploy the proposed models for CIFAR-10 dataset on Raspberry Pi, and more powerful Odroid devices to evaluate several scenarios with homogeneous and heterogeneous devices. We demonstrate 6.22×-12.22× gain in performance and 12.99×-14.36× gain

Table 1. Comparison to Prior Art

Area	Model Compression	Communication-Aware	Distributed Inference	Complements our work
Quantization [8, 12]	✓	✗	✗	✓
Pruning [4, 9, 13, 23]	✓	✗	✗	✓
Separable convolutions [3, 19, 26]	✓	✗	✗	✓
KD [1, 6, 25]	✓	✗	✗	✗
SplitNet [10]	✗	✓	✓	✗
MoDNN, DeepThings [14, 28]	✗	✓	✓	✓
Proposed NoNN	✓	✓	✓	✓

NoNN is the most complete approach as it considers communication costs while performing model compression for distributed inference on IoT.

in per node energy w.r.t. teacher. Moreover, for distributed inference on multiple edge devices, NoNN results in up to 33× speedup in total latency w.r.t. a prior model compression baseline [25]. We also show that the proposed approach is robust and achieves high accuracy even with a reduced number of devices.

The rest of the paper is organized as follows: Section 2 reviews relevant literature and background. The impact of splitting a deep network horizontally on distributed inference performance is discussed in Section 3. Section 4 describes our proposed approach, while Section 5 presents our detailed experimental results. Section 6 describes a case study on hardware deployment of NoNN. The paper is finally concluded in Section 7 with remarks on future work.

2 RELATED WORK AND BACKGROUND

We first discuss prior art on model compression and distributed inference. The relevant background on Knowledge Distillation (KD), and network science is discussed later in this section.

2.1 Model Compression/Distributed Inference

Table 1 summarizes the key differences between the present work and the existing literature from model compression and distributed inference. Specifically, existing approaches for model compression such as quantization [8, 12], pruning [4, 13, 23], KD [1, 6, 25], and separable convolutions [3, 19, 26] are *not* communication-aware and do *not* address the distributed inference problem. For instance, although pruning is effective at reducing parameters/FLOPS, it does *not* result in highly-parallel model architectures. Clearly, pruned models that cannot fit on a single IoT-device will need to be distributed across multiple devices which will incur heavy communication costs (see Section 3 for details).

On the other hand, existing distributed inference techniques such as SplitNet [10], MoDNN [14], and DeepThings [28] do not exploit *ideas* from model compression (e.g., pruning/distillation, or memory-constraints). Specifically, SplitNet [10] splits the network into disjoint parts during training without any constraints on individual network partitions. This unconstrained splitting can result in large partitions which may not conform to memory-budgets of individual IoT devices. Hence, taking specific memory budgets into account is more effective (and necessary) in order to meet hard memory-constraints of IoT-devices; such constraints are not considered by SplitNet [10]. Moreover, by partitioning the feature activation maps of convolutions, MoDNN [14] and DeepThings [28] successfully reduce the FLOPS and the feature activation map memory during distributed inference. However, both of these approaches assume that the mobile device is big enough to fit the entire deep learning model, which is a strong assumption and, hence, they do not

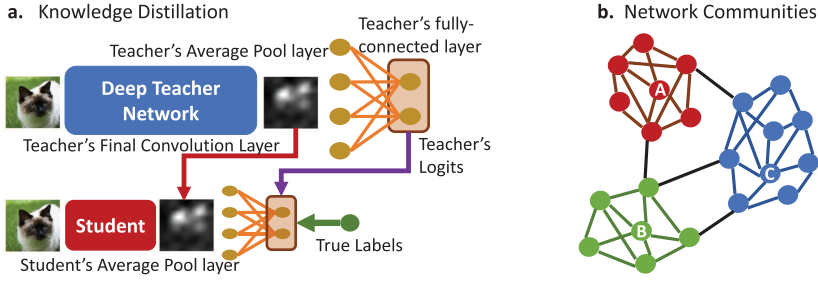


Fig. 2. (a) Knowledge Distillation (KD) is based on a significantly smaller student model trained to mimic a large teacher network. (b) Network communities and hub nodes. In Section 4, we will explain how teacher's activations from (a) can be used to create a network similar to that shown in (b).

address the memory due to model-weights. In contrast, we take the distributed inference one step further to an even more constrained IoT environment (e.g., 500KB memory) where the physical IoT-devices cannot fit a single model. In such cases, the model itself must be split in non-intuitive ways which can result in heavy communication cost at every layer. Therefore, in this paper, our proposed NoNN compresses the model (to reduce the memory and computation costs such that individual modules fit on the devices) while accounting for communication among the student nodes.

Note that, most of the prior techniques above are complementary to our work so they can be used synergistically on top of our approach. For instance, another recent work called MeDNN [15] extends MoDNN with a pruning technique to reduce the per-node computation. Since pruning can still be performed on top of our proposed NoNN and because both MeDNN and MoDNN partition the feature maps (and not the weights), our work is complementary to MeDNN [15]. We also show that we can achieve higher accuracy than several KD baselines and SplitNet in our experiments, i.e., the only two techniques in Table 1 that do not complement our work.

2.2 Knowledge Distillation Background

KD, as shown in Figure 2(a), consists of two deep networks: (i) *Teacher* is the large deep network which we want to compress, and (ii) *Student* is a significantly smaller neural network which is trained to mimic the output of the teacher network. To mimic the teacher, the simplest way is to directly train a student model on teacher's *logits* (see Figure 2(a)) – the unnormalized outputs of the teacher model before the softmax – instead of training on true labels from the dataset [1]. The basic idea here is that while training the student model, using logits instead of direct classification probabilities can transfer more information about the teacher to the student. Hinton et al. [6] argued that we must not only train the student model on the correct predictions made by the teacher, but must also quantify how wrong the teacher was about the incorrect classes. Hence, Hinton et al. introduced KD [6] which uses both the *hard-label loss* (based on true labels from the dataset), as well as the *soft-label loss* (based on logits) to train a significantly smaller student model.

Mathematically, let l_T and l_S be the logits of teacher and student respectively, and τ is a temperature parameter (see [6]), y be the true labels, and P_T^τ and P_S^τ respectively denote the softmax over relaxed logits l_T/τ and l_S/τ . Then, the KD loss ($\mathcal{L}^{kd}$) is given by:

$$\mathcal{L}^{kd}(\theta_S) = (1 - \alpha)\mathbb{H}(y, P_S) + \alpha\mathbb{H}(P_T^\tau, P_S^\tau) \quad (1)$$

where, $\mathbb{H}$ is the standard cross-entropy loss, θ_S denotes the parameters of the student network, and α controls the weight of hard-label loss vs. soft-label loss. The temperature parameter in the second term of Equation (1) improves knowledge transfer from the teacher to the student. Other variants of KD like Attention Transfer-based KD (ATKD) further use intermediate outputs of

teacher's convolution layers while training the student [25]. Of note, KD-based techniques have also been proposed for RNNs [11, 21]. However, our focus in this paper is on KD-based distributed inference for CNNs.

2.3 Network Science Background

Network science has emerged for problems such as modeling information diffusion in social networks, epidemics, transportation, *etc.* [16]. However, to our knowledge, we are the first to utilize network science concepts for model compression. Below, we describe the core ideas from network theory used in this paper.

2.3.1 Node Degree/Hubs. Given an undirected network $\mathcal{G}$ with nodes $\mathcal{V}$, links $\mathcal{E}$, and the adjacency matrix $\mathcal{F} = \{F_{ij}\}$ describing link weights between any two nodes $i, j \in \mathcal{V}$. Then, degree k_i of a node i refers to total number of links connected to node i . A network can have a *scale-free* structure where some nodes have many connections but most nodes have low degree [16]; such nodes with many connections in scale-free networks act as *hubs* of information. To illustrate, Figure 2(b) shows nodes A, B, and C as examples of hubs in a network. As evident, these nodes are characterized by more connections than other nodes in the network.

2.3.2 Community Structure. Many real world networks are organized into groups of tightly connected nodes known as the *community structure* [17]. For instance, in a social network, a community can refer to a group of users with common interests like sports or politics. Formally, a community can be defined as a group of nodes for which the number of connections within the group is significantly higher than what one would expect at random. Figure 2(b) shows the community structure in a network. In contrast to older graph partitioning techniques, community detection automatically partitions the network into its natural groupings without requiring us to specify predefined parameters such as number/size of communities. Size and number of communities in a network instead depend on a user-specified *resolution* parameter γ .

3 MOTIVATION

We now discuss the impact of horizontally splitting a Convolutional Neural Network (CNN) on distributed inference performance. More precisely, when a CNN does not fit on a single memory-constrained device, it must be split “horizontally” for parallel execution on multiple devices (for better utilization of all resources). However, as mentioned in Section 1, this can result in significant communication costs when each split is deployed on a separate device. To verify this assumption, we conduct the following experiment.

We start with a large Wide Resnet (WRN40-4) teacher model with 8.9 million parameters, trained on CIFAR-10 dataset¹ [24]. This model takes about 86ms for inference on a single powerful x86 machine. Next, we split the WRN40-4 model horizontally (e.g., if a layer has 32 channels, we split it into two parts with 16 channels each). We then deploy the individual horizontal splits on two powerful x86 machines connected via a wired point-to-point connection. As shown in Figure 3, since each successive convolution layer requires input from all the output channels from the previous layer, we need to make the output from all devices available to all other devices. Clearly, this results in a significant communication overhead. This is why, even though the computation happens on two powerful x86 machines, the inference time increases **from 86ms to 1006ms (> 10× increase in latency)**. Hence, this experiment shows that, when a deep network does not fit on an IoT-device,

¹The base WRN architecture consists of three groups: G_0 , G_1 , and G_2 with width (i.e., #channels) in each group as [16, 32, 64] respectively. Width multiplier is used to increase the number of channels per group [24]; WRN40-4 implies 40 layers and a width multiplier of 4 which results in [64, 128, 256] channels per group (see Figure 5(a)).

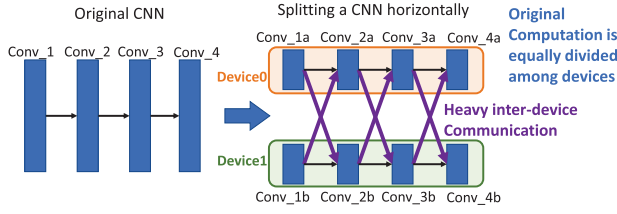


Fig. 3. Splitting a deep network horizontally leads to huge communication cost at every step since the next layer convolutions require access to all the channels at the last layer. Original computation is equally divided between the two devices.

splitting it horizontally is simply not an option. Obviously, the problem gets exacerbated for edge devices which often operate on low frequencies and, hence, take even longer for computation.

As a result, a new memory- and communication-aware model compression technique is imperative for distributed inference on IoT-devices. Therefore, we next describe our proposed approach.

4 PROPOSED APPROACH

We first formulate our problem (Section 4.1), followed by our proposed solution which consists of two stages: (1) Network science-based knowledge partitioning of the teacher: We find disjoint partitions of teacher's knowledge which lead to independent students with minimal communication costs (Section 4.2, Figure 4(b)). (2) Proposed NoNN architecture and the training process: We first select an efficient model architecture for each student that reduces its memory and FLOPS, and then jointly train individual NoNN students on disjoint partitions from teacher (Section 4.3; Figure 4(c)).

4.1 Problem Formulation

In this section, we formulate teacher's knowledge partitioning subject to the resource-constraints of individual students.

As shown in Figure 4(a), the output of final convolution (fconv) layer of teacher (T_{fconv}) gets average-pooled and passes through the fully-connected (fc) layer to yield *logits*. The logits then pass through the softmax layer to generate probabilities. Since our goal is to break teacher's learned function into multiple disjoint parts, we focus on teacher network's fconv. Specifically, we partition the teacher's fconv layer by looking at the *patterns of activation of filters* as validation images pass through the teacher (see Section 4.2).

Partitioning a teacher's fconv often uncovers a partition P_0 which does not contribute to teacher's validation accuracy. Therefore, we should find and remove the largest set of filters $P_0 \subset T_{fconv}$ that does not contribute to teacher's accuracy since such filters only transfer noise to the students during distillation (Section 4.2). Moreover, the sizes of partitions of T_{fconv} should be almost equal so that the resulting student networks fit within a memory budget $\mathcal{B}_{mem}$, while their computation costs fit a FLOP budget $\mathcal{B}_{FLOP}$. Let $h: \mathbb{R} \rightarrow \mathbb{R}$ compute the memory of student i depending on its partition size $|P_i|$, and $f: \mathbb{R} \rightarrow \mathbb{R}$ compute the resulting FLOPS. Then, the *knowledge-partitioning problem* is expressed as:

$$\begin{aligned}
 & \min_{\mathcal{P}=\{P_0, P_1, \dots, P_k\}} |\Delta_{val}| - |P_0| \\
 & \text{subject to} \quad |P_1| \approx |P_2| \dots \approx |P_k|, P_i \sim \text{Rule } R, P_i \subset T_{fconv} \\
 & \quad \quad \quad \forall i \in \{0, \dots, k\} \\
 & \quad \quad \quad |\Delta_{val}| < \epsilon \\
 & \quad \quad \quad \max(h(|P_1|), h(|P_2|), \dots, h(|P_k|)) < \mathcal{B}_{mem} \\
 & \quad \quad \quad \max(f(|P_1|), f(|P_2|), \dots, f(|P_k|)) < \mathcal{B}_{FLOPS}
 \end{aligned} \tag{2}$$

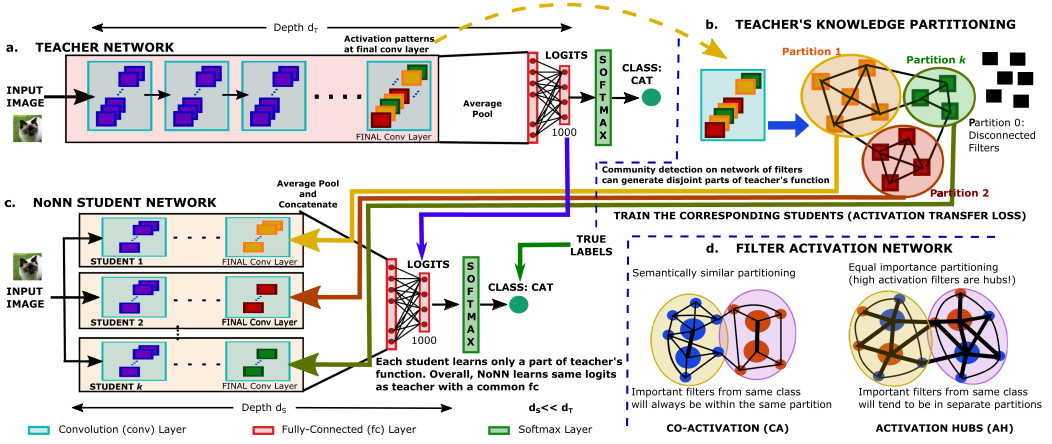


Fig. 4. Complete flow of our approach. (a) The pretrained teacher model exhibits certain activation patterns for the validation set. (b) We use these activation patterns to build a network of filter activity. Community detection on the filter network partitions teacher's knowledge into simpler functions. (c) Train individual students in NoNN to mimic each partition. (d) Teacher's knowledge can be partitioned in different ways depending on how the filter activation network is built (e.g., via Co-Activation (CA) rule or Activation Hubs (AH) rule). Node colors represent filters relevant to a given class, whereas node sizes represent importance of a filter (determined by activation value).

where, $|\Delta \text{val}|$ denotes the absolute value of change in validation accuracy due to removal of P_0 filters. Also, $|P_i|$ does not denote the absolute value, but rather the number of elements in the set P_i (i.e., the cardinality of P_i). The first constraint in (2) aims to keep the sizes of all partitions (except P_0) almost equal since the sizes of student models must be below the *fixed* memory budget $\mathcal{B}_{mem}$. The objective function and the second constraint aim to minimize change in teacher's validation accuracy while removing as many useless filters from T_{fconv} as possible. The last two constraints in problem (2) specify the per-device memory- and FLOP-budgets that must be satisfied by each student (which will be deployed on individual devices). Note that, while it is always possible to satisfy memory- and FLOP-constraints, there will be a tradeoff between model-size or computation and the NoNN accuracy. For instance, for fixed number of students, larger individual student modules can lead to higher overall accuracy. Finally, the rule R needed to determine how the partitions are formed is described below.

4.2 Network Science-Based Knowledge Partitioning: Filter Activation Network

We propose a network science-based solution to the knowledge partitioning problem. The idea is to uniformly distribute knowledge from teacher's fconv using network science in order to jointly train individual students in the NoNN.

To partition teacher's fconv, we first build a network $\mathcal{F} = \{F_{ij}\}$ of filter activation patterns, where i and j are two filters. Now, let a_i denote the *average activity* of a filter i for a given image in the validation set (val). Average activity of a filter i is defined as the averaged output of the corresponding output channel of teacher's fconv. For instance, suppose a teacher network's fconv has 256 output channels, and height and width of its activation map is 8×8 pixels. Then, to obtain average activity of filter i , we average the value of its 8×8 output channel activation map. Essentially, we use this average activity metric as a measure of *importance* of a filter for a given class of images; that is, the higher the average activity of a filter, the more important it is for classification for some class of images. In Figure 4(d), each circle depicts a filter at teacher's fconv, and filters

shown in same color activate for similar classes. Also, the bigger the size of the circle, the more important is the filter for the corresponding class (i.e., it has higher average activity).

To create the filter activation network, there exist multiple strategies (i.e., rules R) by which the filters can be connected. Figure 4(d) illustrates two such strategies: (i) **Activation Hubs (AH)**: Encourage connections between very important and less important filters (i.e., less number of high-importance filters get surrounded by many low-importance filters and thus act as hubs), and (ii) **Co-Activation (CA)**: Filters that activate together for similar classes get connected. The former rule encourages partitions of roughly equal importance (since each partition will have a few important filters). The latter rule partitions the filters into semantically similar parts. Formally,

$$F_{ij} = \begin{cases} \sum_{\text{val}} a_i a_j |a_i - a_j| & \text{AH Rule} \\ \sum_{\text{val}} \frac{a_i a_j}{|a_i - a_j| + 1} & \text{CA Rule} \end{cases} \quad (3)$$

Let us analyze what happens during the AH case. If i and j are two filters with average activities a_i and a_j , respectively, the F_{ij} link weight for AH case will be high if, for a given image, either a_i is high and a_j is low, or vice versa. Note that, if either a_i or a_j is close to zero, that link is not created. Further, when a_i and a_j are both very high for the same image (i.e., both filters are important for same class), then that link will also not be encouraged (since the link weight will be low due to the $|a_i - a_j|$ part). Therefore, important filters from the same class will be discouraged from connecting together. Consequently, for AH Rule, highly important filters (with high a_i) will be encouraged to connect with not-so-important filters (from same or other classes). Important filters from same classes will be forced to occupy separate communities; hence, the knowledge is distributed uniformly across the students. Similar discussion holds for CA Rule where the communities represent semantically-similar features. Of note, this flexibility in partitioning teacher's f_{conv} comes from network science ideas.

Of course, there can be other ways filters may be connected. Indeed, if the dataset contains too many semantically similar classes, CA can lead to heterogeneous student models (because many filters from $T_{f_{\text{conv}}}$ will go into a few partitions). Hence, to obtain student models of similar sizes, we only explore AH in this paper. In matrix form, adjacency matrix of AH network can be written as follows:

$$\mathcal{F}^{AH} = \sum_{n \in \text{val}} a_n a_n^T \odot |\mathbf{D}_n - \mathbf{D}_n^T|, \quad (4)$$

where, a_n is the vector of average filter activities (a_i 's) for each image $n \in \text{val}$, matrix $\mathbf{D}_n$ contains all columns as a_n , and $\odot$ denotes element-wise multiplication. To partition this AH network, we need to detect communities by maximizing a modularity function as explained in the network science literature [17]:

$$\max_{\mathbf{g} = \{g_0, g_1, \dots, g_{l-1}\}} \frac{1}{2m} \sum_{ij} \left[F_{ij}^{AH} - \frac{1}{\gamma} \cdot \frac{k_i k_j}{2m} \right] \delta(g_i, g_j), \quad (5)$$

where, m is #edges, k_i is degree (number of connections) of node i , resolution γ controls the size/number of communities, and δ is Kronecker delta. The idea is to find groups of tightly connected nodes, $\mathbf{g} = \{g_0, g_1, \dots, g_{l-1}\}$, which map the nodes $\mathcal{V}$ to l communities. Finally, these l communities are converted to k partitions P_i by combining the communities such that the partition sizes are almost equal (so that resulting students stay within the budget). This partitioning $\mathcal{P}$ is then used to train individual student networks. Of note, removing community g_0 (which consists of all disconnected nodes in the filter network) often has very little impact on teacher's accuracy. Hence, partition $P_0 = g_0$ is obtained directly by community detection. Consequently, we solve problem (2) by detecting and removing the largest community g_0 that does

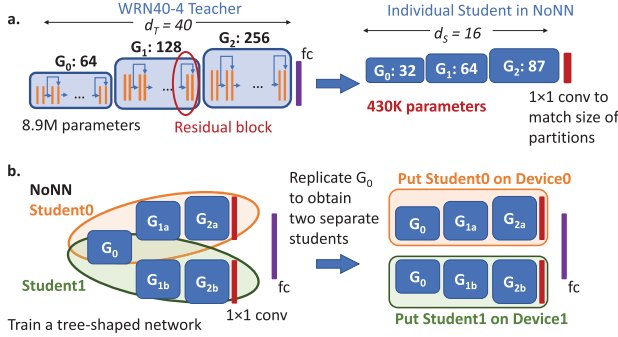


Fig. 5. (a) Selecting an individual NoNN student architecture: Reduce depth and channel-width so that it fits within the desired budgets. (b) Overall, NoNN trains a tree-shaped network (where possible) with some initial layers common (denoted as G_0). At inference time, replicate the common group G_0 and put individual students on separate devices.

not change teacher's validation accuracy, and then combine the rest of the communities to form the remaining partitions. For instance, say we have four remaining communities after removing g_0 : $\{|g_1| = 12, |g_2| = 14, |g_3| = 25, |g_4| = 30\}$ ($|x|$ indicates size of group x), and we want to combine them into two partitions of nearly equal sizes. Then, we can combine them as $\mathcal{P} = \{|P_1| = |g_2 \cup g_3| = 39, |P_2| = |g_1 \cup g_4| = 42\}$. Hence, we obtain the required k almost equal partitions from l communities.

4.3 Network of Neural Networks (NoNN)

Once we have partitioned the teacher's fconv, we jointly train our student networks on individual partitions from T_{fconv} as shown in Figure 4(c). However, we must first make the following design decisions: (i) Select the deep network architecture for individual students, and (ii) Select how students are connected together in NoNN.

4.3.1 Student Architecture Selection. Similar to prior art in KD [6, 25], we select our individual student networks to be significantly reduced versions of the teacher (i.e., far fewer layers and lower width). Specifically, we pick our individual student models based on user-defined memory- and FLOP-constraints for IoT-devices. For instance, for CIFAR-10 dataset, our teacher is Wide Resnet model WRN40-4 with 40 layers and width multiplier of 4. That is, as shown in Figure 5(a) (left), it has 64 channels in first group, 128 in second group, and 256 in third group. Also, suppose that our each individual student must have less than 500K parameters. Then, we select our individual student (see Figure 5(a) (right)) with 16 layers and channel-width appropriately adjusted so that each student has less than 500K parameters. Note that, student networks are not exactly the same as they mimic different partitions. Therefore, we need an additional 1×1 convolution layer to make the dimensions equal between the teacher's knowledge partition and individual student's output layer. Next, we describe the architecture of NoNN containing multiple such students, and the training process.

4.3.2 NoNN Architecture. A NoNN consisting of two students is shown in Figure 5(b). As evident, wherever possible, we make the initial few layers from the students common (since the knowledge learned at initial few layers will be common for all students). Moreover, at inference time, the common layers between all students can be simply replicated across multiple devices without significant increase in memory per student (since the common group is not too big). Once the common group is replicated, the individual students are completely independent and do not

communicate until the final fc layer (see Figure 5(b)). Hence, we obtain a final NoNN architecture which consists of multiple disjoint students. The output from all students is concatenated and passed through a fc to yield logits.

4.3.3 Training Loss. To train the overall NoNN, we use the KD loss (see Equation (1)) and, as given below, we additionally propose a new loss function called *activation-transfer* for transferring partitioned knowledge from the teacher to individual students. The main motivation behind the activation-transfer loss function is that each individual student must mimic the corresponding partition of teacher's knowledge. Specifically, for each partition of teacher's knowledge, we want to minimize the error between activations of the teacher's filters (that belong to the given partition) and activations of filters in the corresponding student. We define the activation-transfer loss function as given below:

$$\mathcal{L}^{Act}(\theta_S) = \sum_{p \in \mathcal{P}} \left\| \frac{v_T^F(p)}{\|v_T^F(p)\|} - \frac{v_S^F(p)}{\|v_S^F(p)\|} \right\|_2^2 \quad (6)$$

where, $v_Z^F(p) = \text{vec}(A_Z^F(p))$, $Z \in \{T, S\}$ is the vectorized fconv activations for partition $p \in \mathcal{P}$ of teacher or for individual student network in NoNN, and the $\|v_Z^F(p)\|$, $Z \in \{T, S\}$ terms denote the partition-wise normalization. Hence, the total loss is given by $\mathcal{L} = \mathcal{L}^{kd}(\theta_S) + \beta \mathcal{L}^{Act}(\theta_S)$ which can be minimized via stochastic gradient descent.

This completes our proposed NoNN. We next present detailed experimental evaluation for NoNN.

5 EXPERIMENTAL SETUP AND RESULTS

In this section, we first present our experimental setup and then results for different datasets and NoNN architectures.

5.1 Experimental Setup

We compress various deep networks for five well-known image classification tasks: CIFAR-10, CIFAR-100, Scene [18], Caltech-UCSD-Birds (CUB) [22], and Imagenet. Scene and CUB datasets belong to the transfer learning domain where a pretrained model is finetuned on a different problem. For comparison, we consider strong teacher-student baselines like KD [6], Attention-transfer with KD (ATKD) [25]. For KD and ATKD, we use models from the WRN family [24] as single, large students. Also, *total parameters* in NoNN (i.e., parameters in all students combined) are comparable to the parameters in KD/ATKD baseline models. We also show that we outperform prior models like Splitnet [10]. Further, we show experiments with up to 8 student networks for CIFAR-10/100. With these experiments, we thoroughly demonstrate that NoNN compresses the teacher model into disjoint subsets which do not communicate until the fc layer.

We first train the teacher model on 90% of training dataset while saving 10% for validation. This validation set is used for building the filter activation network to partition the teacher's knowledge. Then, we use the Activation Transfer loss given in Equation (6) and KD-loss to train the compressed NoNN model. Moreover, all the accuracies reported in the paper are on test set for the respective datasets. We set the number of partitions, $k = 2$. To build more than two students, since our objective is to keep the individual student architectures as similar as possible, we simply shuffle the filters in these two partitions while transferring knowledge to rest of the students. This ensures that the size of students is not too diverse and limits the possible factors that can contribute to the improvement in accuracy of our model. Also, we set $\alpha = 0.9$ and selected the best $\beta = \{10^1, 10^2, \dots, 10^4\}$ for both ATKD [25] and NoNN. For training NoNN, we set the initial

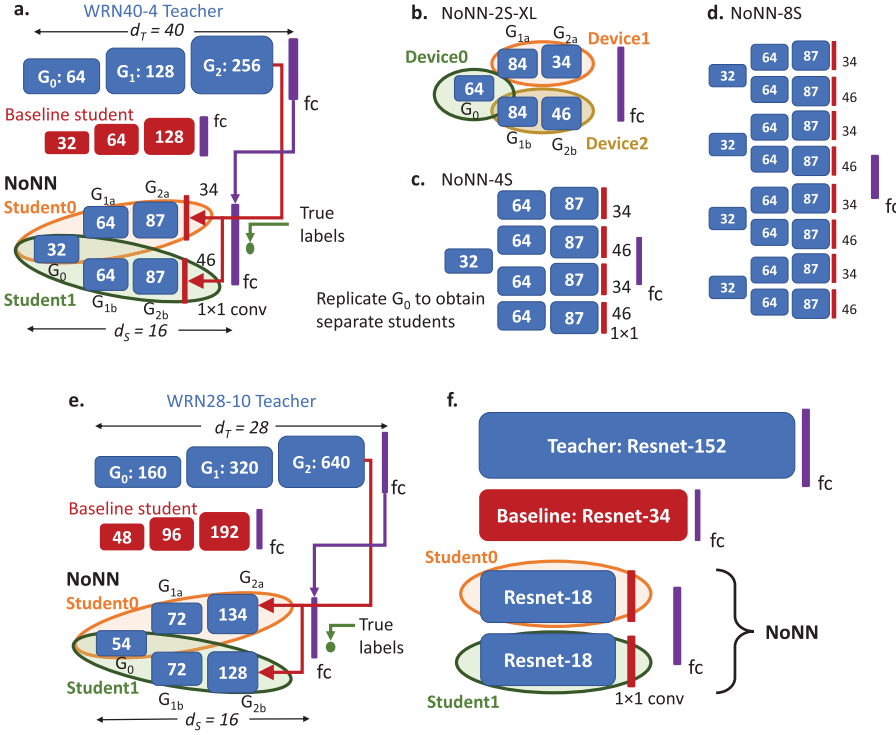


Fig. 6. Teacher, baseline student, and NoNN models for various datasets. (a) WRN40-4 teacher, WRN-16-2 baseline, and 2-student NoNN for CIFAR-10. (b) A larger 2-student NoNN (NoNN-2S-XL) can be distributed on three separate devices to keep FLOPs under a budget. (c, d) 4-student and 8-student NoNNs. (e) CIFAR-100 models, (f) Transfer learning models. After replicating the common groups (if any), the individual students in our proposed NoNN will communicate only at fc layer.

learning rate as 0.1 and used a momentum of 0.9. Finally, our entire framework is implemented in Pytorch which is run on NVIDIA GTX 1080-Ti and Titan Xp GPUs.

5.2 Results

We first show results on CIFAR-10/100 and transfer learning datasets (Scene, CUB), and then our preliminary results for Imagenet.

5.2.1 CIFAR-10. Figure 6(a) depicts our teacher model for CIFAR-10 as WRN40-4 and our baseline student for KD and ATKD as WRN16-2. As evident, knowledge from partitions of only 34 and 46 filters from teacher's fconv is transferred to student-0 and student-1, respectively (via 1×1 convolutions). The rest 176 out of 256 filters at teacher's fconv do not contribute to teacher's accuracy and, hence, form partition P_0 (or community g_0) in $\mathcal{F}^{AH}$; this partition is not used for transferring knowledge to NoNN students. We set $\mathcal{B}_{FLOPS} \approx 200M$ which is close to baseline FLOPs used by WRN16-2 model. Further, based on our initial motivation w.r.t. Arm Cortex-M, $\mathcal{B}_{mem} = 0.5M$ parameters (since for models quantized to 8-bits, 0.5M parameters mean less than 500KB of memory).

As shown in Table 2, our NoNN-2S model (Figure 6(a)) achieves higher accuracy than the baseline student model. Note that, each student in NoNN utilizes fewer FLOPs (i.e., 167M FLOPs) than the allowed budget. Moreover, although *each* student has 0.43M parameters, due to common G_0 , total parameters for NoNN-2S in Figure 6(a) is 0.82M (comparable to 0.7M in WRN16-2). Further,

Table 2. CIFAR-10 Teacher-Student Results*

Model	#parameters (largest student)	#FLOPS (largest student)	Accuracy
Teacher WRN40-4	8.9M	2.6G	95.49%
KD WRN16-2 [6]	0.7M	202M	93.79 $\pm$ 0.15%
ATKD WRN16-2 [25]	0.7M	202M	93.83 $\pm$ 0.08%
NoNN-2S	0.43M**	167M	94.32 $\pm$ 0.17%
KD: NoNN-2S-XL	0.46M	245M	93.96 $\pm$ 0.09%
NoNN-2S-XL	0.46M**	245M***	94.53 $\pm$ 0.19%

*For statistical significance, results are reported as mean $\pm$ standard deviation of five experiments.

**All NoNN #parameters/FLOPS are reported for one student only. Complete NoNN has very similar number of parameters as baselines for fair comparison. Our contribution is to show that NoNN can be broken down into disjoint parts that stay below certain budgets and do not communicate until the last layer.

***NoNN-2S-XL can be distributed onto three devices as shown in Figure 6(b).

after replicating G_0 , student-0 and student-1 will not communicate with each other until the final fc layer. Clearly, when quantized, our individual student models of 430K parameters can fit on a device with 500KB memory (430K parameters are after replicating G_0). Indeed, if a device cannot fit more than 500K parameters, prior models such as WRN16-2 (with 700K parameters) will need to be distributed across multiple devices which will lead to communication at every layer; that is, when distributed, the baseline WRN16-2 will communicate at all 16 layers, whereas NoNN communicates only at the final layer.

This situation gets exacerbated by higher accuracy baseline student models (e.g., 40-layer WRN40-2) for which communication costs at every layer grow rapidly. In contrast, as we shall see shortly, our student models can be very flexible: we can have larger students (see Figure 6(b)), or many disjoint students without significantly increasing the communication costs (e.g., see NoNN-4S and NoNN-8S in Figure 6(c, d)). Concrete energy and latency for CIFAR-10 experiments are reported in the hardware deployment described in Section 6.

For an even higher per-device FLOP budget (say, 250M), a NoNN-2S-XL student architecture can be distributed on three devices (see Figure 6(b)). Although the larger student in this case has 0.46M parameters ($G_0 \rightarrow G_{1b} \rightarrow G_{2b}$), due to a bigger common G_0 group, total parameters for NoNN-2S-XL is 0.77M (again, comparable to 0.7M in WRN16-2). Also, each individual student in this model has higher FLOPS because the common group G_0 is wider and has 245M FLOPS. Since it is impractical to replicate this group, as shown in Figure 6(b), we can assign a completely new device (device-0) to this group and put $G_{1a} \rightarrow G_{2a}$ ($G_{1b} \rightarrow G_{2b}$) on device-1 (device-2).

Table 2 demonstrates that our NoNN-2S-XL achieves even better accuracy (<1% away from the teacher). Moreover, this increase in accuracy is not only due to an increased FLOP budget. To demonstrate, we show that filter network community-based knowledge transfer from teacher plays a more significant role in accuracy improvement. If we simply train our NoNN-2S-XL student model via KD, i.e., without the $\mathcal{L}^{Act}$ loss defined in Equation (6), the accuracy increases only slightly over the baseline WRN16-2 (e.g., 93.96% vs. 93.79%). Therefore, community-based knowledge partitioning is extremely important for successfully training our proposed NoNN.

Next, we show that NoNN is scalable to many datasets and model sizes for different memory and FLOP budgets.

5.2.2 CIFAR-100 and Comparison with SplitNet. Table 3 shows results for CIFAR-100. Here, the teacher is a WRN28-10 model with 36.5M parameters and our baseline student is WRN16-3 model

Table 3. CIFAR-100 Teacher-Student Results*

Model	#parameters (largest student)	#FLOPS (largest student)	Accuracy
Teacher WRN28-10	36.5M	10.48G	79.28%
KD WRN16-3 [6]	1.5M	446M	73.99 $\pm$ 0.26%
ATKD WRN16-3 [25]	1.5M	446M	74.90 $\pm$ 0.17%
NoNN-2S	0.85M	345M	75.74 $\pm$ 0.31%

*For statistical significance, results are reported as mean $\pm$ standard deviation of five experiments.

Table 4. Accuracy for More Students*

#S	CIFAR-10						CIFAR-100		
	NoNN	ΔM^{**}	ΔF^{**}	NoNN-XL	ΔM	ΔF	NoNN	ΔM	ΔF
2	94.32 $\pm$ 0.17%	10 $\times$	7.7 $\times$	94.53 $\pm$ 0.19%	11 $\times$	5 $\times$	75.74 $\pm$ 0.31%	24 $\times$	15 $\times$
4	94.64 $\pm$ 0.16%	5.1 $\times$	3.9 $\times$	94.87 $\pm$ 0.14%	6.3 $\times$	3.3 $\times$	77.07 $\pm$ 0.18%	12 $\times$	7.6 $\times$
6	94.90 $\pm$ 0.06%	3.5 $\times$	2.6 $\times$	95.02 $\pm$ 0.10%	3.9 $\times$	1.7 $\times$	77.25 $\pm$ 0.24%	7.7 $\times$	5 $\times$
8	95.02 $\pm$ 0.08%	2.5 $\times$	2 $\times$	95.28 $\pm$ 0.07%	2.9 $\times$	1.3 $\times$	77.03 $\pm$ 0.17%	5.7 $\times$	3.8 $\times$
T	95.49%	1 $\times$	1 $\times$	95.49%	1 $\times$	1 $\times$	79.28%	1 $\times$	1 $\times$

*For statistical significance, results are reported as mean $\pm$ standard deviation of five experiments.

** ΔM (ΔF) shows compression rate, i.e., Δ Memory (Δ FLOPS) for *complete* NoNN (i.e., parameters of all students combined) w.r.t. teacher.

containing 1.5M parameters (see Figure 6(e)). Therefore, we create a NoNN with overall 1.5M parameters as shown in Figure 6(e); this NoNN can be split into two students where the larger student has 0.85M parameters (after replicating G_0). Also, the FLOPS for our larger student are lower than the WRN16-3 baseline. Again, NoNN-2S model is found to be more accurate than KD and ATKD, while creating disjoint students that can fit within some memory budget (say, 1MB). Once our disjoint students fit within the 1MB budget, techniques like MoDNN [14] can further distribute our model's computation on more devices with similar memory-constraints to reduce the FLOPS. MoDNN assumes that the model fits on the device, and our work makes it possible to meet the memory-constraints.

We next compare our model to SplitNet [10] that splits a deep network into multiple disjoint parts (albeit without any model compression ideas such as resource-constraints). Kim et al. obtained two CIFAR-100 models via SplitNet: (i) A 7.42M parameter WRN model with 2 sub-groups which achieves 76.04% accuracy, and (ii) A 4.12M parameter model with 4 sub-groups which achieves 75.2% accuracy. For fair comparison with SplitNet, we create two corresponding models: First with 2-students (7.42M parameters), and the second with 4-students (4.13M parameters), respectively. While keeping the experimental setup same as SplitNet (e.g., same validation set size, etc.), we achieve **79.07% accuracy** for the 7.42M parameter model, and **77.42% accuracy** for the 4.13M parameter model. Therefore, the proposed NoNN significantly outperforms SplitNet for similar model-sizes. Moreover, the NoNN-2S model shown in Table 3 which has merely 1.5M total parameters (i.e., 0.85M parameters per student) achieves higher accuracy than the 4.13M parameter SplitNet model.

5.2.3 Varying Number of Students. Now, we vary students from two to eight for CIFAR datasets. As evident from Table 4, NoNN models achieve close to teacher's accuracy while reducing memory/FLOPS by orders of magnitude (i.e., 2.5 $\times$ -24 $\times$ reduction in #parameters and 2 $\times$ -15 $\times$ reduction in FLOPS w.r.t. teacher). Specifically, the NoNN-8S model in Figure 6(d) achieves

Table 5. Transfer Learning Results*

Model	#parameters (largest student)	#FLOPS (largest student)	Accuracy		ΔM^{**}	ΔF^{**}
			Scene	CUB		
Teacher Resnet-152	58M	11G	79.44%	80.94%	1×	1×
KD Resnet-34 [6]	22M	4G	$76.92 \pm 0.39\%$	$78.59 \pm 0.22\%$	2.5×	2.8×
ATKD Resnet-34 [25]	22M	4G	$77.63 \pm 0.94\%$	$79.10 \pm 0.56\%$	2.5×	2.8×
NoNN-2S	11M	2G	$77.48 \pm 0.76\%$	$79.81 \pm 0.33\%$	2.5×	2.8×

*For statistical significance, results are reported as mean $\pm$ standard deviation of five experiments.

** ΔM (ΔF) shows compression rate, i.e., Δ Memory (Δ FLOPS) for *complete* NoNN (i.e., parameters of all students combined) w.r.t. teacher.

95.02% accuracy (i.e., less than 0.5% away from teacher) while using eight separate students (after G_0 is replicated), each of which can fit within 167M FLOPS and 430K parameters (i.e., <500KB when quantized to 8-bits), and do not communicate until the final fc layer. Overall, this results in $2600/(167 \times 8) \approx 2\times$ reduction in FLOPS, and $8.9/(0.43 \times 8) \approx 2.5\times$ reduction in parameters over the teacher.

Finally, to compare NoNN-8S for CIFAR-10, we also experimented with a larger WRN40-2 student (40 layers, 2.2M parameters, and 655M FLOPS) with ATKD [25]. Although, this model achieves similar accuracy of 95.03%, it clearly does *not* result in an architecture that can be distributed on multiple devices (since it will require 2.2MB storage when quantized to 8-bits). As a result, distributing this single model on multiple devices will lead to heavy inter-device communication at each of the 40 layers (see Section 3), whereas our model will communicate only at the final layer. This clearly highlights the significance of our proposed NoNN.

5.2.4 Transfer Learning Datasets. We now demonstrate the effectiveness of NoNN for transfer learning tasks where the idea is to finetune a model pretrained on Imagenet w.r.t. a new dataset. Table 5 shows results for two such datasets: (i) Indoor Scene Classification, (ii) Caltech-UCSD Birds. For both datasets, we used Resnet-152 as teacher model, Resnet-34 as baseline student, and our NoNN consists of two separate Resnet-18 models with a common fc layer (see Figure 6(f)). As evident from Table 5 and Figure 6(f), our NoNN-2S model (total 23M parameters) consists of disjoint parts of 11M parameters each, which do not communicate until the last layer. Also, for Scene dataset, NoNN-2S achieves slightly lower (but still comparable) accuracy than ATKD on Resnet-34. However, NoNN-2S achieves much higher accuracy than prior approaches for CUB.

5.2.5 Preliminary Imagenet Results. We used the fastai setup [7] to obtain imagenet results in a fast and inexpensive way. Our teacher model is Resnet-34 with 73% top-1 and 91% top-5 accuracy. We used Resnet-18 model (11.68M parameters) as a baseline ATKD student which achieved 69.98% top-1 and 89.50% top-5 accuracy. Our NoNN consists of two students (each with around 6M parameters; total 11.69M parameters). For training our NoNN, we use attention transfer losses [25] on *each* student since activation transfer loss led to some underflow problems². Even with this setup, our NoNN achieved *comparable* accuracies, i.e., 69.82% top-1, and 89.61% top-5 while ensuring that the two students do not communicate until the last layer. With availability of more resources, we should be able to improve these results even further.

So far, we have shown that NoNN achieves higher accuracy than several baselines, and minimal communication among students. We have also demonstrated that NoNN achieves close to teacher's

²Fastai uses a floating point-16 (fp16) format for computation on 8 NVIDIA Volta GPUs. Due to an underflow problem on fp16 (which is well-known to happen with fp16 format), we couldn't use activation transfer $\mathcal{L}^{Act}$ losses for training our NoNN.

accuracy with $2.5\times$ – $24\times$ lower memory, and $2\times$ – $15\times$ fewer FLOPS than the teacher model. Next, we deploy our models on real edge devices and show significant gain in performance and energy.

6 CASE STUDY: HARDWARE DEPLOYMENT

In this section, as a case study, we deploy our NoNN models on two hardware-constrained devices to quantify their effectiveness in terms of latency and energy on real hardware.

6.1 Hardware Setup

We implement NoNN on edge devices such as Raspberry Pi (RPI) and Odroid-XU4S. We demonstrate an extensive exploration of two scenarios that can benefit from our approach: (i) Homogeneous case in which all devices are identical, and (ii) Heterogeneous case, where devices have different resources and computing power. We perform detailed evaluation of accuracy, performance, and energy for each scenario. We show these results for our teacher (WRN40-4), NoNN-2S, and NoNN-8S models trained on the CIFAR-10 dataset.

We use eight Raspberry Pi-3 Model-B and eight Odroid-XU4S boards. Each RPi has an Arm Cortex-A53 quad-core processor with nominal frequency of 1.2GHz and 1GB of RAM. We scaled the frequency of RPi's down to 400MHz as the RPi's running at maximum frequency demonstrated unstable behavior and would often crash due to high temperature. The XU4S boards have a Samsung Exynos5422 SoC, which has an Arm big.LITTLE architecture with 4 big Cortex-A15 and 4 little Cortex-A7 cores, and 2GB of RAM. The Odroid is executed at the nominal frequency: 2GHz for the big cores and 1.4GHz for the little cores. To maximize performance, we run the inference on the big cores in the Odroids.

In order to measure voltage, current, and power, we use the Odroid Smart Power 2 for the Odroid-XU4S and AVHzy USB Power Meter for the RPi's. In addition, we use an x86 machine (Core i7), further referred as *server*, to send the images to the devices, concatenate the results, and apply the fc layer. Of note, the fc is small and could also be placed in another edge/mobile device. We perform all of our experiments with a point-to-point wired local network connecting all devices together.

Since the NoNN models were built with PyTorch, we use TVM³, a deep learning compiler that optimizes NN models for several hardware architectures, to deploy the trained model into the RPi's and Odroid boards. To achieve this, the PyTorch models are converted into an intermediate representation with Open Neural Network Exchange (ONNX) [2]. Then the ONNX representation is used by TVM to generate the binary that is deployed in the target device. In addition, we integrate these binaries into a TCP-wrapper so that each student can be distributed to separate devices and the communication is performed through the TCP protocol.

6.2 Hardware Results

We first show the results for NoNN when all devices are identical (i.e., the homogeneous case). Then, we address the heterogeneous scenario, where one device is more powerful than the rest. We also discuss the case when some devices are unavailable for inference.

6.2.1 Homogeneous Devices. Table 6 presents the evaluation of NoNN-2S, NoNN-8S, and teacher models for both RPi's and Odroids. In this experiment, each student is deployed on a different device, so in the 8-student case we have eight separate devices (either RPi or Odroid). From the experiment in Section 3, since splitting the teacher model horizontally leads to more than $10\times$ increment in latency (even on powerful x86 machines), we executed the teacher on a single edge device, as otherwise it would take even longer.

³TVM Compiler: <https://tvm.ai/>.

Table 6. Accuracy, Performance, and Energy Results for CIFAR-10 on Both Raspberry Pi and Odroid-XU4S

	Raspberry Pi					Odroid-XU4S				
	Teacher	2S	8S	Improvement		Teacher	2S	8S	Improvement	
				2S	8S				2S	8S
Accuracy	95.49%	94.32%	95.02%	-1.17%	-0.47%	95.49%	94.32%	95.02%	-1.17%	-0.47%
Latency (ms)	1405	115	150	12.22×	9.37×	224	25	36	8.96×	6.22×
Energy per node (mJ)	3430.67*	238.98	249.15	14.36×	13.77×	3084.08*	219.07	237.45	14.08×	12.99×
Theoretical FLOPS per student	2.6G*	167M	167M	15.56×	15.56×	2.6G*	167M	167M	15.56×	15.56×

*Teacher model is executed on a single node since splitting the teacher model onto multiple devices incurs heavy communication latency (see Section 3). NoNN energy and FLOP numbers are reported per student because we are mostly concerned with per node budgets for memory and FLOPS. Of note, 2S (8S) refers to the NoNN-2S (NoNN-8S) model with two (eight) individual students deployed on two (eight) separate devices.

Table 7. Average Latency Breakdown Per Inference

Latency (ms)	Raspberry Pi		Odroid-XU4S		x86
	2S	8S	2S	8S	2-Split Teacher
Computation	101.91	106.24	19.01	20.60	86
Communication	13.10	43.76	6.00	15.40	920

On the RPi's, NoNN-2S improves the performance and energy per node by 12.22× and 14.36× w.r.t. teacher. Also, since NoNN-8S synchronizes eight devices, it achieves 9.37× better latency and 13.77× better energy per node. The same trend is observed for the Odroid boards, having 8.96× and 14.08× improvement for NoNN-2S in performance and energy per node, respectively, and 6.22× and 12.99× for NoNN-8S. Note that, the reduction in theoretical FLOPS for NoNN w.r.t. the teacher is about 15×, while energy per node reduction for both RPi's and Odroids is 13-14×. This shows very good agreement between theory and practice. Further, the latency and energy per node do *not* increase significantly between NoNN-2S and NoNN-8S for both devices. Therefore, our proposed NoNN leads to significant gains in performance and per node energy, while maintaining accuracy within 0.5%-1.17% compared to the teacher.

Table 7 presents the average latency breakdown for computation and communication per image, considering the 2S and 8S cases. The computation time for one image increases by merely 4% and 8% for RPi and Odroid, respectively, when going from 2S to 8S, because each device operates in parallel. But when communication is considered, the overhead of sending and receiving data for 8S increases the average latency by 3.34× for RPi's, and 2.57× for Odroid boards (compared to the 2S-case). Again, this communication cost is very small compared to the cost incurred from directly splitting the single, large models. For instance, we have 43ms communication latency for distributing NoNN-8S on RPi's. In contrast, splitting the teacher on x86 incurs $1006 - 86 = 920$ ms communication latency (see Section 3), which is 21× higher than NoNN-8S.

6.2.2 Comparison of Latency for Distributed Inference w.r.t. a Model Compression Baseline. Next, we present a concrete, side-by-side comparison between the distributed inference latency incurred by our proposed NoNN and ATKD [25], a traditional model compression baseline. Note that, we compare the latency of NoNN against ATKD [25] as it achieves the best baseline accuracy results. Consequently, for this experiment, we use the ATKD model WRN40-2 (2.2M parameters, 95.03% accuracy on CIFAR-10 dataset) distilled from the same teacher model as above (i.e., WRN40-4, 8.9M parameters, 95.49% accuracy). To achieve higher accuracy, the size of the model compressed

Table 8. NoNN and Split-ATKD Latency on RPi's (Pytorch)

	Split-ATKD (WRN40-2)		NoNN-8S
	4 RPi's	8 RPi's	8 RPi's
Accuracy	95.03%	95.03%	95.02%
Parameters per device	550K	275K	430K
FLOPS per device	163M	82M	167M
Total latency per inference (s)	23	28.5	0.85
Speedup with NoNN	27×	33×	—

via ATKD grows significantly and, therefore, it must be distributed across multiple resource-constrained devices as it may not fit within the given per-device memory budget (henceforth known as the *split-ATKD* experiment); similar to the split-teacher experiment above (see Section 3 and Table 7), ATKD baseline incurs communication at each intermediate layer. Since TVM generates a single binary file for the entire deep network, models compiled in TVM cannot be used to perform such split-teacher or split-ATKD experiments on RPi's (as we need to extract the intermediate outputs at each layer and communicate them across devices). Hence, for the following split-ATKD experiments, we install Pytorch 0.4.1 on all eight RPi's to run distributed inference for both split-ATKD and NoNN-8S.

Table 8 shows the accuracy, parameters per device, FLOPS executed per device, and total latency for NoNN-8S and split-ATKD models in pytorch on RPi's. We distribute the ATKD baseline in two different ways:

- (1) **ATKD split on four RPi's (split-ATKD-4S):** Each RPi for this case stores about 550K (2.2M/4) parameters and executes about 163M (655M/4) FLOPS. This experiment simulates the scenario where each device has, say, close to 500K-parameter memory-budget (similar to the 430K parameter-budget considered in NoNN for the CIFAR-10 dataset).
- (2) **ATKD split on eight RPi's (split-ATKD-8S):** Each RPi for this case stores around 275K (2.2M/8) parameters and executes merely 82M (655M/8) FLOPS. Similar to NoNN-8S, this case compares the performance of split-ATKD on eight separate devices.

Note that, the split-ATKD-4S case has slightly more parameters than the single NoNN-8S student (550K vs. 430K) but executes slightly fewer FLOPS than NoNN-8S (163M vs. 167M). On the other hand, the split-ATKD-8S case executes far fewer FLOPS than a single NoNN-8S student (82M vs. 167M) and also stores fewer parameters than NoNN-8S (275K vs. 430K). As evident from Table 8, despite the lower computation involved in both the split-ATKD cases, the heavy communication at each layer incurred by split-ATKD-4S (split-ATKD-8S) results in a total latency of 23s (28.5s). In contrast, NoNN-8S takes only 0.85s (in pytorch) to perform the distributed inference⁴, which is 27× (33×) better than the split-ATKD-4S (split-ATKD-8S) baseline, while achieving nearly the same accuracy. Hence, this result emphasizes the importance of our proposed NoNN for distributed deep learning inference on IoT.

6.2.3 Heterogeneous Devices. To demonstrate NoNN's flexibility, we perform the following experiment: we start with a homogeneous scenario for the RPi's, then, gradually move each student to an Odroid board, depicting an environment where we have several small devices (RPi's) and one

⁴The total latency for distributed inference via NoNN-8S increases from 150ms in TVM (see Table 6) to 850ms in pytorch. This shows that the device-level optimizations performed by TVM are important for deep learning inference on edge devices.

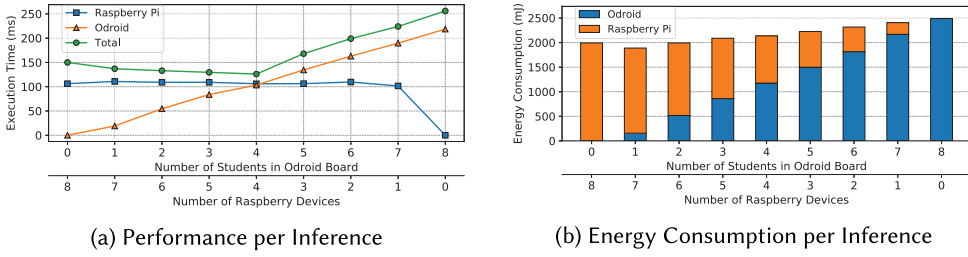


Fig. 7. Performance and energy as the number of Raspberry devices is reduced and the workload for the Odroid board is increased, always executing a total of eight students.

slightly more powerful device (Odroid), the latter running on the big cores to maximize performance. Each small device (RPI) executes only one student, while the more powerful device (Odroid) executes multiple students.

Figure 7(a) breaks down the latency into three parts: latency of RPI's, the single Odroid, and the *total latency* including the server. The total latency represents the execution time, starting when the server transmits the first image and ending when *fc* layer is applied (after receiving and concatenating results from all students). The RPI performance represents the latency (i.e., both computation and communication time) for the *slowest* device.

As the Odroids are considerably faster than the RPI's in the current setup, a single Odroid is able to execute four students in the same amount of time a RPI takes to compute one student (Figure 7(a)). As the number of students in the Odroid increases even further, its latency becomes a bottleneck and the total latency increases. The difference between the total latency and the slowest device is the communication time between the server and the devices, which is 30.65ms on average. The server (x86 machine) takes 0.1ms to concatenate and apply the *fc* layer, which is negligible. Note that, until four students are deployed in Odroid, the total latency in Figure 7(a) keeps on reducing since the server can process the Odroid results while the RPI's are still running the individual students. Hence, performing this task in parallel reduces the critical path.

Figure 7(b) depicts the energy consumption for *all* RPI devices and the single Odroid device. As evident, we have a trade-off among performance, energy, and number of devices. For instance, when the Odroid runs just one student, the total energy gets minimized because Odroid is more energy-efficient than the RPI for the one student case. However, as we increase the workload for the Odroid (i.e., number of students), the total energy increases even though the number of devices (RPI's) decreases. In the current setup, inference on eight separate RPI's consumes the same amount of energy as using six RPI's plus two students on the Odroid. Comparing the two extreme cases (eight students in eight RPI's, or all eight students in a single Odroid), the Odroid consumes 25% more energy.

Therefore, this evaluation demonstrates three important aspects: (i) NoNN approach demonstrates good performance in both homogeneous and heterogeneous environments, (ii) In terms of energy, distributing the students on several devices instead of using a single more powerful device can reduce the total energy consumption for inference. (iii) Finally, to maximize the energy savings for the entire system, deploying NoNN on a combination of several devices can trade-off between energy dissipation and number of devices.

6.2.4 Robustness of NoNN. We now evaluate a scenario where some devices may become unavailable due to a variety of reasons such as power depletion or processor/network failure. We evaluate all possible cases in which such devices may fail. For instance, for six active devices, any two devices may fail, resulting in 28 possible scenarios, and so on for the remaining cases.

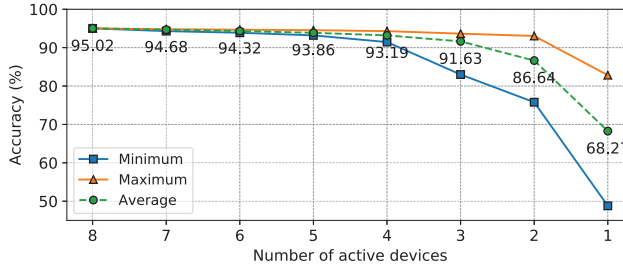


Fig. 8. Accuracy as some devices become unavailable due to device failures (e.g., due to processor or network failures, and battery depletion). NoNN achieves more than 91% accuracy, on average, when only three devices are available. When four devices are available, NoNN achieves more than 93% accuracy.

Figure 8 shows the accuracy drop as we remove some devices. On an average, NoNN is still able to achieve more than 91% accuracy when only three devices are available. Moreover, if a minimum of four devices are available, the average accuracy is more than 93%. As more devices become unavailable, the difference between the minimum and the maximum accuracy increases since the final decision depends on output from all devices. When only a few devices are available, the final result depends on the characteristics of each student, some being able to deliver higher accuracy than others. For instance, one of the students is able to achieve 82.82% accuracy alone (i.e., the maximum point when only one student is active in Figure 8). Therefore, deploying this student on the primary device (which will always be available) will guarantee at least 82.82% accuracy. Other devices can then be used to further improve the accuracy of the system. Hence, NoNN is robust to random device failures. Of note, robustness can also be formulated as part of the partitioning problem itself; this, however, is left as a future work.

7 CONCLUSION AND FUTURE WORK

In this paper, we have proposed a new NoNN paradigm to compress a large teacher model into multiple highly-compressed student modules that can be distributed across a network of edge devices with minimal communication overhead. To this end, we have first proposed a network science-based partitioning of teacher's knowledge, and then trained individual students on the corresponding partitions.

With extensive experiments, we have demonstrated that NoNN achieves close to teacher's accuracy with significantly lower memory (2.5×-24× gain w.r.t. teacher) and computation (2×-15× fewer FLOPS w.r.t. teacher), while guaranteeing that individual modules of NoNN fit within some given memory/FLOP budget. We have also shown that NoNN achieves higher accuracy than several baselines with no communication among students until the last layer.

Finally, we have deployed the proposed models for CIFAR-10 dataset on Raspberry Pi and Odroid, and have demonstrated 6.22×-12.22× improvement in performance and 12.99×-14.36× in energy per node w.r.t. teacher. We have further shown that NoNN model results in up to 33× reduction in total latency for distributed inference on multiple edge devices w.r.t. a state-of-the-art model compression baseline. Hence, the proposed communication-aware model compression can ultimately lead to effective deployment of deep networks on multiple memory-constrained IoT-devices.

The current work opens up several new directions. In future, we plan to address problems like how to accurately map deep network partitions to heterogeneous devices, and also make robustness as a primary objective of NoNN. We further plan to explore NoNN for other types of deep learning models such as RNNs.

ACKNOWLEDGMENTS

This material is based on research sponsored in part by Air Force Research Laboratory (AFRL) and Defense Advanced Research Projects Agency (DARPA) under agreement number FA8650-18-2-7860. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of Air Force Research Laboratory (AFRL) and Defense Advanced Research Projects Agency (DARPA) or the U.S. Government.

We thank the anonymous reviewers for their useful suggestions which helped improve the paper. Also, we thank Prof. Virginia Smith of Carnegie Mellon University for useful discussions. We further thank J. Wang, I. Pardesi, and W. Chen of Carnegie Mellon University for their help with the experimental part. We also thank NVIDIA Corporation for donating a Titan Xp GPU which was used for conducting some experiments. Finally, we acknowledge Amazon support through the AWS ML Research Program, and FastAI (<https://www.fast.ai/>) for releasing code to accelerate training on Imagenet dataset using AWS machines.

REFERENCES

- [1] Jimmy Ba and Rich Caruana. 2014. Do deep nets really need to be deep? In *Advances in Neural Information Processing Systems*. 2654–2662.
- [2] Facebook. 2017. ONNX: Open Neural Network Exchange Format. <https://onnx.ai/>
- [3] Hongyang Gao, Zhengyang Wang, and Shuiwang Ji. 2018. ChannelNets: Compact and efficient convolutional neural networks via channel-wise convolutions. In *Advances in Neural Information Processing Systems*. 5203–5211.
- [4] Song Han, Huizi Mao, and William J. Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv:1510.00149* (2015).
- [5] Song Han, Jeff Pool, John Tran, and William Dally. 2015. Learning both weights and connections for efficient neural network. In *NIPS*. 1135–1143.
- [6] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv:1503.02531* (2015).
- [7] Jeremy Howard. 2018. Imagenet in 18 minutes. <https://www.fast.ai/2018/08/10/fastai-diu-imagenet/>. (2018). Accessed: 2018-10-01.
- [8] Itay Hubara et al. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *JMLR* 18, 1 (2017), 6869–6898.
- [9] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size. *arXiv:1602.07360* (2016).
- [10] Juyong Kim, Yookoon Park, Gunhee Kim, and Sung Ju Hwang. 2017. SplitNet: Learning to semantically split deep networks for parameter reduction and model parallelization. In *International Conference on Machine Learning*. 1866–1874.
- [11] Yoon Kim and Alexander M. Rush. 2016. Sequence-level knowledge distillation. *arXiv preprint arXiv:1606.07947* (2016).
- [12] Liangzhen Lai, Naveen Suda, and Vikas Chandra. 2017. Deep convolutional neural network inference with floating-point weights and fixed-point activations. *arXiv:1703.03073* (2017).
- [13] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2016. Pruning filters for efficient convnets. *arXiv:1608.08710* (2016).
- [14] Jiachen Mao et al. 2017. Modnn: Local distributed mobile computing system for deep neural network. In *2017 DATE Conference*. IEEE, 1396–1401.
- [15] Jiachen Mao, Zhongda Yang, Wei Wen, Chunpeng Wu, Linghao Song, Kent W. Nixon, Xiang Chen, Hai Li, and Yiran Chen. 2017. Mednn: A distributed mobile system with enhanced partition and deployment for large-scale dnns. In *Proceedings of the 36th International Conference on Computer-Aided Design*. IEEE Press, 751–756.
- [16] Mark Newman, Albert-Laszlo Barabasi, and Duncan J. Watts. 2011. *The Structure and Dynamics of Networks*. Vol. 19. Princeton University Press.
- [17] Mark E. J. Newman. 2006. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences* 103, 23 (2006), 8577–8582.
- [18] Ariadna Quattoni and Antonio Torralba. 2009. Recognizing indoor scenes. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 413–420.

- [19] Mark Sandler et al. 2018. Inverted residuals and linear bottlenecks: Mobile networks for classification, detection and segmentation. *arXiv:1801.04381* (2018).
- [20] STMicro. 2018. Datasheet for Arm-Based Microcontroller with up to 512KB total storage (including FLASH memory). Product Page: <https://bit.ly/2I5ZSMR>. Datasheet. <https://bit.ly/2Kz8ehD>
- [21] Zhiyuan Tang, Dong Wang, and Zhiyong Zhang. 2016. Recurrent neural network training with dark knowledge transfer. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 5900–5904.
- [22] Peter Welinder, Steve Branson, Takeshi Mita, Catherine Wah, Florian Schroff, Serge Belongie, and Pietro Perona. 2010. Caltech-UCSD birds 200. (2010).
- [23] Tien-Ju Yang et al. 2016. Designing energy-efficient convolutional neural networks using energy-aware pruning. *arXiv:1611.05128* (2016).
- [24] Sergey Zagoruyko and Nikos Komodakis. 2016. Wide residual networks. *BMVC* (2016).
- [25] Sergey Zagoruyko and Nikos Komodakis. 2017. Improving the performance of convolutional neural networks via attention transfer. *ICLR* (2017).
- [26] Xiangyu Zhang et al. 2017. ShuffleNet: An extremely efficient convolutional neural network for mobile devices. *CoRR* abs/1707.01083 (2017).
- [27] Yundong Zhang, Naveen Suda, Liangzhen Lai, and Vikas Chandra. 2017. Hello edge: Keyword spotting on microcontrollers. *arXiv:1711.07128* (2017).
- [28] Zhuoran Zhao, Kamyar Mirzazad Barijough, and Andreas Gerstlauer. 2018. DeepThings: Distributed adaptive deep learning inference on resource-constrained IoT edge clusters. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018), 2348–2359.

Received April 2019; revised June 2019; accepted July 2019